```
001    i12 = _y;                                          /* i12 points to beginning of y[ ] */
002    i4 = _x;                                           /* i4 points to beginning of x[ ] */
003
004    f2 = dm(i4,m6), f4 = pm(i12,m14)                   /* prime the registers */
005    f8 = f2*f4, f2 = dm(i4,m6), f4 = pm(i12,m14);
006
007    lcntr = 18, do (pc,1) until lce;                   /* highly efficient main loop */
008    f12 = f8 + f12, f8 = f2*f4, f2 = dm(i4,m6), f4 = pm(k12,m14);
009
010    f12 = f8 + f12, f8 = f2*f4;                        /* complete the last loop */
011    f12 = f8 + f12;
012
013    dm(_result) = f12;                                 /* store the result in memory */
```

TABLE 28-4
Dot product in assembly (optimized). This is an optimized version of the program in TABLE 28-2, designed to take advantage of the SHARC's highly parallel architecture.

and i12 are pointed to the starting locations of these arrays. Next, we execute 20 loop cycles, as controlled by line 004. The format for this statement takes advantage of the SHARC DSP's *zero-overhead looping* capability. In other words, all of the variables needed to control the loop are held in dedicated hardware registers that operate in parallel with the other operations going on inside the microprocessor. In this case, the register: *lcntr* (loop counter) is loaded with an initial value of 20, and decrements each time the loop is executed. The loop is terminated when *lcntr* reaches a value of zero (indicated by the statement: *lce*, for "loop counter expired"). The loop encompasses lines 004 to 008, as controlled by the statement (pc,4). That is, the loop ends four lines after the current program counter.

Inside the loop, line 005 loads the value from *x*[ ] into data register f2, while line 006 loads the value from *y*[ ] into data register f4. The symbols "dm" and "pm" indicate that the values are fetched over the "data memory" bus and "program memory" bus, respectively. The variables: i4, m6, i12, and m14 are registers in the data address generators that manage the circular buffers holding *x*[ ] and *y*[ ]. The two values in f2 and f4 are multiplied in line 007, and the product stored in data register f8. In line 008, the product in f8 is added to the accumulator, data register f12. After the loop is completed, the accumulator in f12 is transferred to memory.

This program correctly calculates the dot product, but it does not take advantage of the SHARC highly parallel architecture. Table 28-4 shows this program rewritten in a highly optimized form, with many operations being carried out in parallel. First notice that line 007 only executes 18 loops, rather than 20. Also notice that this loop only contains a single line (008), but that this line contains multiple instructions. The strategy is to make the loop as efficient as possible, in this case, a single line that can be executed in a single clock cycle. To do this, we need to have a small amount of code to "prime" the